

Hands-On TensorFlow/Keras



“TensorFlow is Google’s open-source library for numerical computation and machine learning. **Keras** is its *high-level API* — a user-friendly front end that lets us build, train, and evaluate neural networks with just a few lines of code.”

Keras sits on top of TensorFlow and hides most of the low-level math and graph handling.

“API stands for *Application Programming Interface* — it’s the set of functions and classes through which we talk to TensorFlow.

Keras gives us three major APIs:

- “**Sequential API** → stack layers one after another (simple linear flow)”
- “**Functional API** → connect layers flexibly (graphs, branches, skip-connections)”
- “**Model Subclassing API** → write custom forward passes for research work.””

Prerequisite

- Deep Learning Basics
- Optimization
- Hyperparameters
- Bias-variance – Overfitting-Underfitting
- Case-study on classification & Regression

Programming for DL

Step 1: Creating the setup

- Create Project folder
- Open [VS Code or Jupyter notebook](#)
- Open Anaconda Prompt or CMD prompt (from VS code)
- Create an environment (e.g. tf_projects_env)
- Activate the environment
- Install required packages on the environment

TensorFlow, CUDA Toolkit, scikit-learn, Keras, Pandas, etc.

```
# =====
```

``` # TensorFlow + Keras (GPU) Environment Setup ```

```
# =====
```

```
# Create a new Conda environment with Python 3.10
```

```
conda create -n tf_keras_coding_env python=3.10 -y
```

```
# Activate the environment
```

```
conda activate tf_keras_coding_env
```

```
# Install TensorFlow with GPU support (version 2.13)
```

```
# TensorFlow 2.13 works with CUDA 11.8 + cuDNN 8.6
```

```
pip install tensorflow==2.13
```

```
# Install compatible NVIDIA CUDA toolkit and cuDNN libraries
```

```
# This ensures GPU kernels are properly accessible inside the environment
```

```
conda install -c nvidia cuda-toolkit=11.8 cudnn=8.6 -y
```

```
# Install common ML and visualization packages
```

```
pip install numpy pandas scikit-learn matplotlib jupyterlab
```

```
# Verify GPU detection
```

```
python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

```
# =====  
# TensorFlow + Keras (CPU-only) Environment Setup  
# =====  
  
# Create a new Conda environment with Python 3.10  
conda create -n tf_keras_coding_env python=3.10 -y  
  
# Activate the environment  
conda activate tf_keras_coding_env  
  
# Install CPU-only TensorFlow (no CUDA/cuDNN required)  
pip install tensorflow-cpu==2.13  
  
# Install essential ML and visualization libraries  
pip install numpy pandas scikit-learn matplotlib jupyterlab  
  
# Verify TensorFlow installation  
python -c "import tensorflow as tf; print(tf.__version__);  
print(tf.config.list_physical_devices())"
```

Step 2... you need to know how to define

- Upload a dataset and preprocess it as per requirement

We will not focus on data preprocessing here

- DL architecture and then Compile it (what to define here?)
- Training algorithm → Optimization step (what to define here?)
- Visualize the training process (why?)
- Evaluate on test data
- Decide what to do next...

Classifying movie reviews: A binary classification example

IMDB Movie Reviews Dataset, a binary sentiment classification dataset.

It contains **movie reviews** taken from the Internet Movie Database (IMDB) and each review is labeled as either:

2. Dataset Size and Structure

- 1 → Positive review
- 0 → Negative review
- Total samples: 50,000 reviews
 - 25,000 for training
 - 25,000 for testing
- Each review is already **preprocessed and encoded as integers** (tokens).
- Each integer represents a **word index** from a predefined vocabulary.

Review example:

"The movie was fantastic, absolutely loved it!"

IMDB Tiny Dataset (10 Samples)

review	label
"The movie was fantastic! I loved the characters and the story."	1
"Absolutely terrible film. I wasted two hours of my life."	0
"A brilliant performance by the lead actor. Truly inspiring!"	1
"Not worth watching. Poor script and weak acting."	0
"I really enjoyed this movie, the visuals were stunning."	1

Text → Number (Encoding)

Before training a neural network, text must be converted into numeric form.

```
"The movie was fantastic, absolutely loved it!"
```

```
[1, 14, 20, 45, 678, 120, 84, 9]
```

Steps

1. Load the dataset containing text reviews and labels.
2. Split the dataset into training and testing subsets.
3. Initialize a text tokenizer and limit the vocabulary to the 10 000 most frequent words.
4. Fit the tokenizer on the training reviews to build the word-index mapping.
5. Convert all reviews into one-hot encoded vectors based on the vocabulary (each review becomes a 10 000-length binary vector).
6. Verify that every review now has the same length (10 000).
7. Use these fixed-length vectors as inputs to a Dense Neural Network.
8. Compile the model with an appropriate optimizer, loss function, and metric.
9. Train the model on the one-hot encoded training data.
10. Evaluate the model's performance on the test data.

Load the dataset containing text reviews and labels.

```
from tensorflow.keras.datasets import imdb

# Load IMDB dataset with the top 10,000 most frequent words
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)

# Display dataset information
print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
print("Number of classes:", len(set(y_train)))

# Example: first review (as sequence of word indices)
print("Example review indices:", x_train[0])
print("Label:", y_train[0])
```

```
# Display dataset information
print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
print("Number of classes:", len(set(y_train)))

# Example: first review (as sequence of word indices)
print("Example review indices:", x_train[0])
print("Label:", y_train[5])
```

Training samples: 25000

Testing samples: 25000

Number of classes: 2

Example review indices: [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5]

Label: 0

Step 2: Preprocess the IMDB data for one-hot encoding

```
# Initialize a tokenizer matching the same vocabulary size
tokenizer = Tokenizer(num_words=10000)

# Convert sequences of word indices to one-hot encoded binary matrices
x_train_ohe = tokenizer.sequences_to_matrix(x_train, mode="binary")
x_test_ohe = tokenizer.sequences_to_matrix(x_test, mode="binary")

print("Training shape:", x_train_ohe.shape)
print("Testing shape:", x_test_ohe.shape)
```

How the Tokenizer works

Imagine you have three short sentences:

1. *"I love deep learning"*
2. *"Deep learning loves me"*
3. *"I love AI"*

Step 1 — Build Vocabulary

The tokenizer first **reads all sentences** and notes every unique word. Then it assigns an **index number** to each word, starting from 1.

Example vocabulary:

1 → I
2 → love
3 → deep
4 → learning
5 → loves
6 → me
7 → AI

Step 2 — Convert Text to Sequences

Next, each sentence is converted into a **list of numbers**, where each number represents a word.

Example:

- "I love deep learning" → [1, 2, 3, 4]
- "Deep learning loves me" → [3, 4, 5, 6]
- "I love AI" → [1, 2, 7]

Step 3 — Create a Vector (Optional)

If you want all sentences to be the same length, the tokenizer can also turn them into **fixed-length vectors**, for example:

- Length = 10
- Each vector position corresponds to a word in the vocabulary.
- The value is 1 if that word appears in the sentence, otherwise 0.

So "I love deep learning" becomes something like:

```
[1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
```

We can interact with Keras through either **Sequential-style API** or **Functional-API**

Keras Sequential API = simplest way to build neural networks layer by layer, in order, for straightforward architectures.

```
from tensorflow.keras import models, layers

model = models.Sequential([
    layers.Input(shape=(10000,)),      # fixed-length one-hot input
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid") # binary sentiment
])
```

What is the Functional API?

The **Functional API** lets you build **any kind of neural network architecture**, not just simple stacks.

Unlike the **Sequential API**, it allows:

- Multiple **inputs** and **outputs**
- **Branching** and **merging** layers
- **Shared layers** and **skip connections** (like in ResNet)

It's called "functional" because you literally **call layers as functions** on tensors.

The **Functional API** gives you full control — it's perfect when your model isn't just a straight stack of layers but a **graph of connected components**.

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model

# 1. Define input layer
inputs = Input(shape=(100,))

# 2. Define hidden layers
x = Dense(32, activation='relu')(inputs)
x = Dense(16, activation='relu')(x)

# 3. Define output layer
outputs = Dense(1, activation='sigmoid')(x)

# 4. Create the model
model = Model(inputs=inputs, outputs=outputs)
```

```
from tensorflow.keras.layers import concatenate

inputA = Input(shape=(32,))
inputB = Input(shape=(16,))

# Two branches
x1 = Dense(64, activation='relu')(inputA)
x2 = Dense(32, activation='relu')(inputB)

# Merge them
merged = concatenate([x1, x2])

# Output
output = Dense(1, activation='sigmoid')(merged)

model = Model(inputs=[inputA, inputB], outputs=output)
```



Sequential API Version (Simpler Linear Stack)

python

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

# Build a simple fully connected network
model_seq = Sequential([
    Dense(64, activation='relu', input_shape=(10000,)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_seq.summary()
```

**2**

Functional API Version (Explicit Connections)

python

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model

# Define inputs and connect layers manually
inputs = Input(shape=(10000,))
x = Dense(64, activation='relu')(inputs)
x = Dense(32, activation='relu')(x)
outputs = Dense(1, activation='sigmoid')(x)

model_func = Model(inputs=inputs, outputs=outputs)
model_func.summary()
```

`model.summary()`

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	640,064
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

Total params: 642,177 (2.45 MB)

Trainable params: 642,177 (2.45 MB)

Non-trainable params: 0 (0.00 B)

The compilation step is where you tell the model how to learn — you define three key things before training:

1. Optimizer

- Decides *how the weights will be updated* based on the loss gradient.
- Examples: `adam`, `sgd`, `rmsprop`.
- Analogy: the *learning rule* or “driver” that navigates toward the best solution.

2. Loss Function

- Measures *how wrong* the model's predictions are.
- Guides the optimizer in minimizing error.
- Examples:
 - `binary_crossentropy` → for 2-class problems
 - `categorical_crossentropy` → for multi-class
 - `mse` → for regression.

3. Metrics

- Extra numbers you want to monitor during training and validation (e.g., `accuracy`, `precision`, `recall`).

Compilation = configuring the model's learning process.

```
model.compile(optimizer="adam",  
              loss="binary_crossentropy",  
              metrics=["accuracy"])
```

We use a training function that makes your neural network actually learn from data.

What `.fit()` Does

When you call:

```
python
```

```
model.fit(x_train, y_train, ...)
```

you are telling Keras to **iterate through your training data multiple times and update the model's weights to minimize the loss.**

So `.fit()` = the full training loop →

Forward pass → Loss computation → Backpropagation → Weight update

```
history = model.fit(x_train_subset, y_train_subset ,  
                    validation_split=0.2,  
                    epochs=20,  
                    batch_size=512,  
                    verbose=1)
```

It returns a History object containing loss and metric values per epoch.

What Happens Internally

1. Batch Selection

Training data is split into mini-batches of size = `batch_size`.

2. Forward Pass

Each batch is passed through the network to generate predictions.

3. Loss Computation

The loss function (defined at `compile`) measures prediction error.

4. Backward Pass (Backpropagation)

The optimizer computes gradients of loss w.r.t. weights.

5. Weight Update

Model parameters are updated to reduce the loss.

6. Repeat for All Batches

→ 1 epoch = all training data processed once.

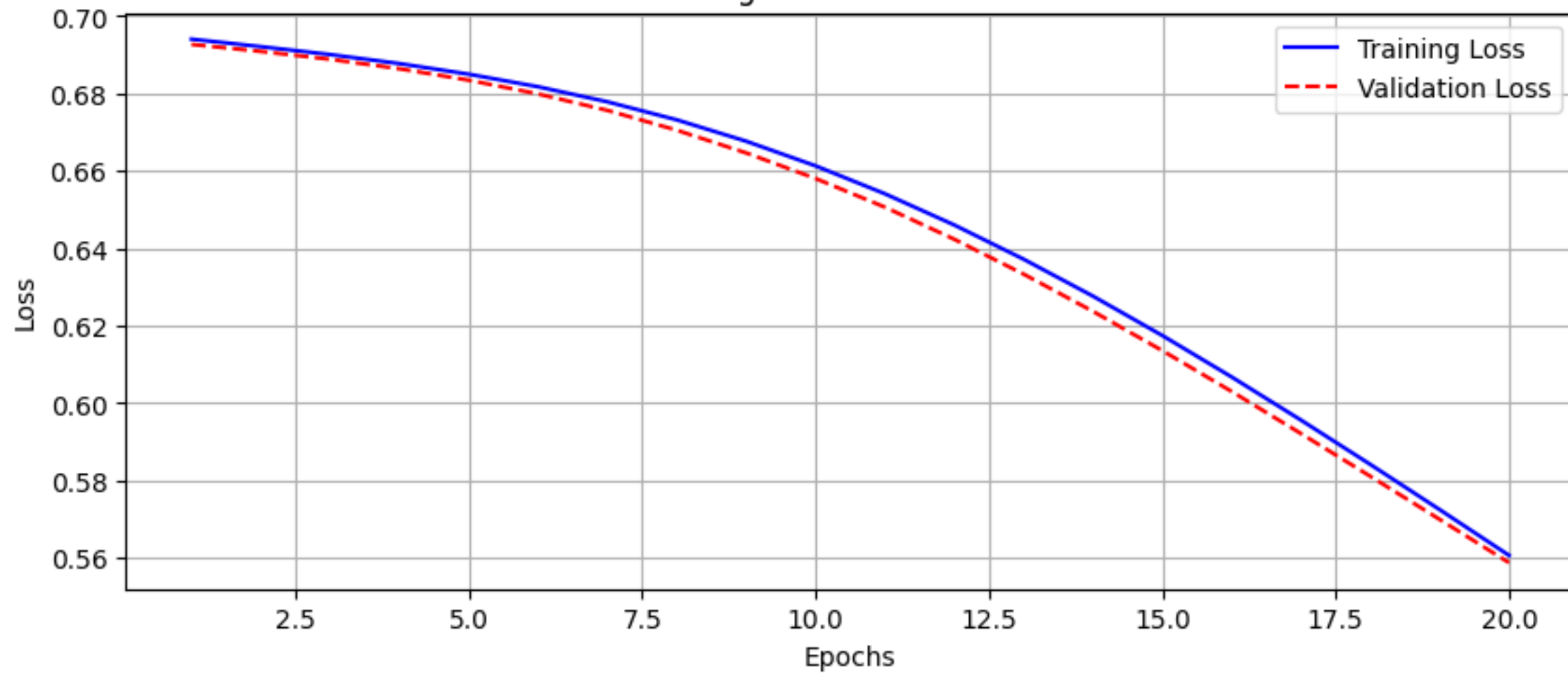
→ Training continues for multiple epochs.

python

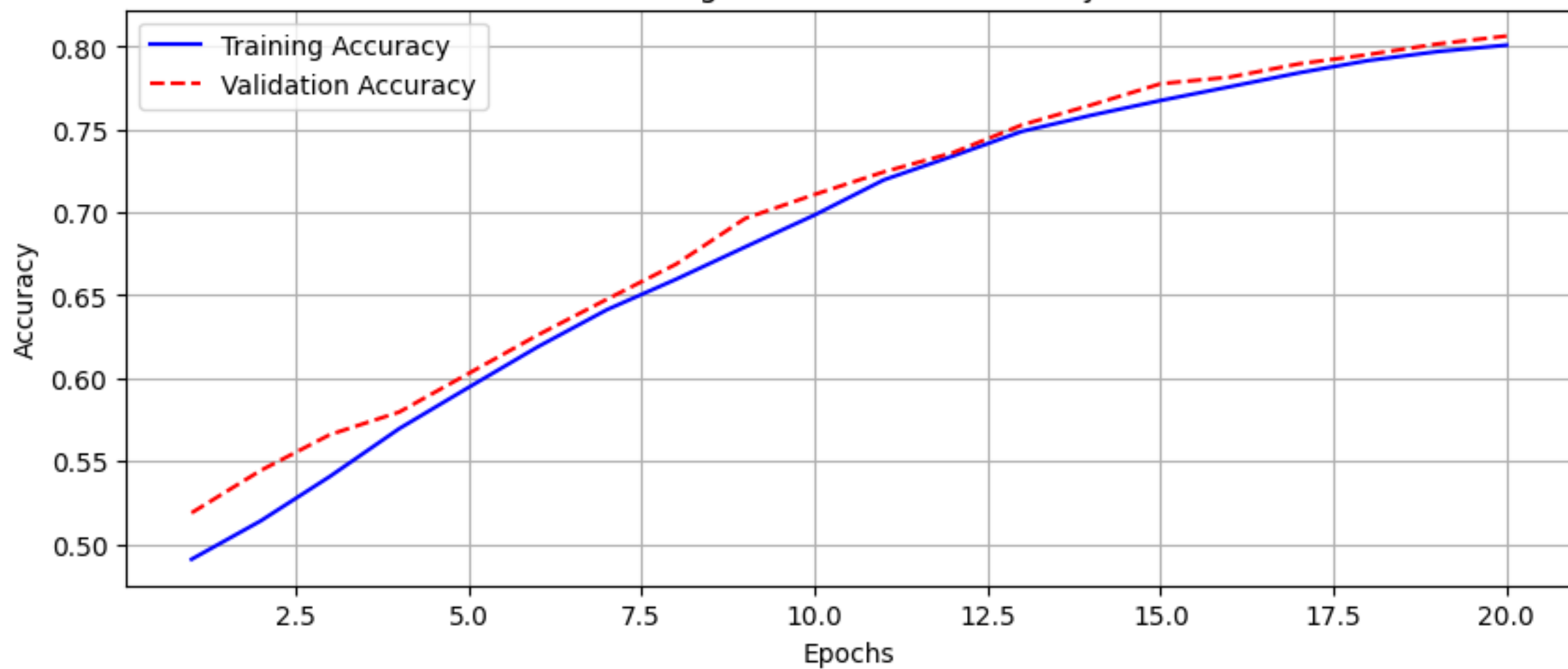
```
loss, acc = model.evaluate(x_test_ohe, y_test, verbose=0)  
print(f"Test accuracy: {acc:.3f}")
```

Test accuracy: 0.792

Training and Validation Loss



Training and Validation Accuracy

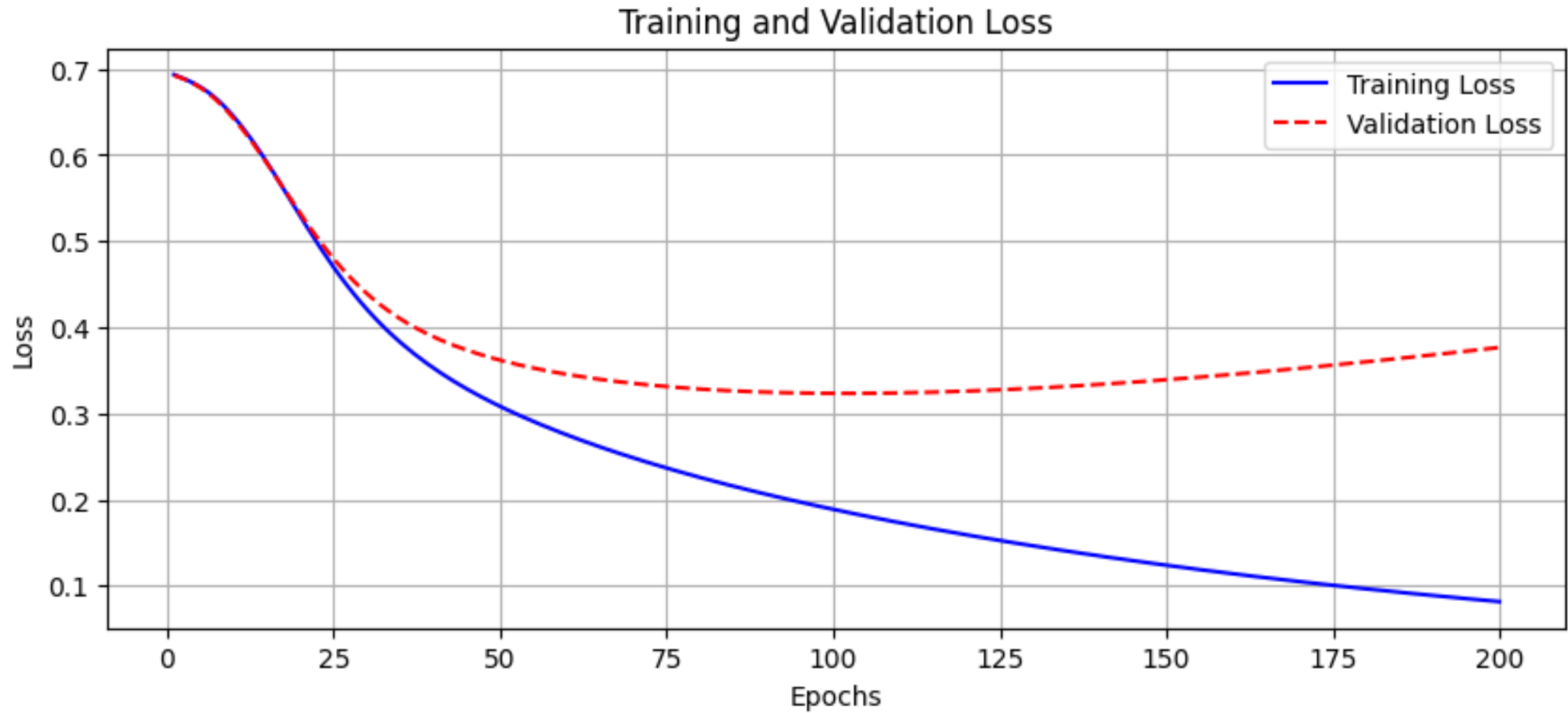


Concluding remarks...

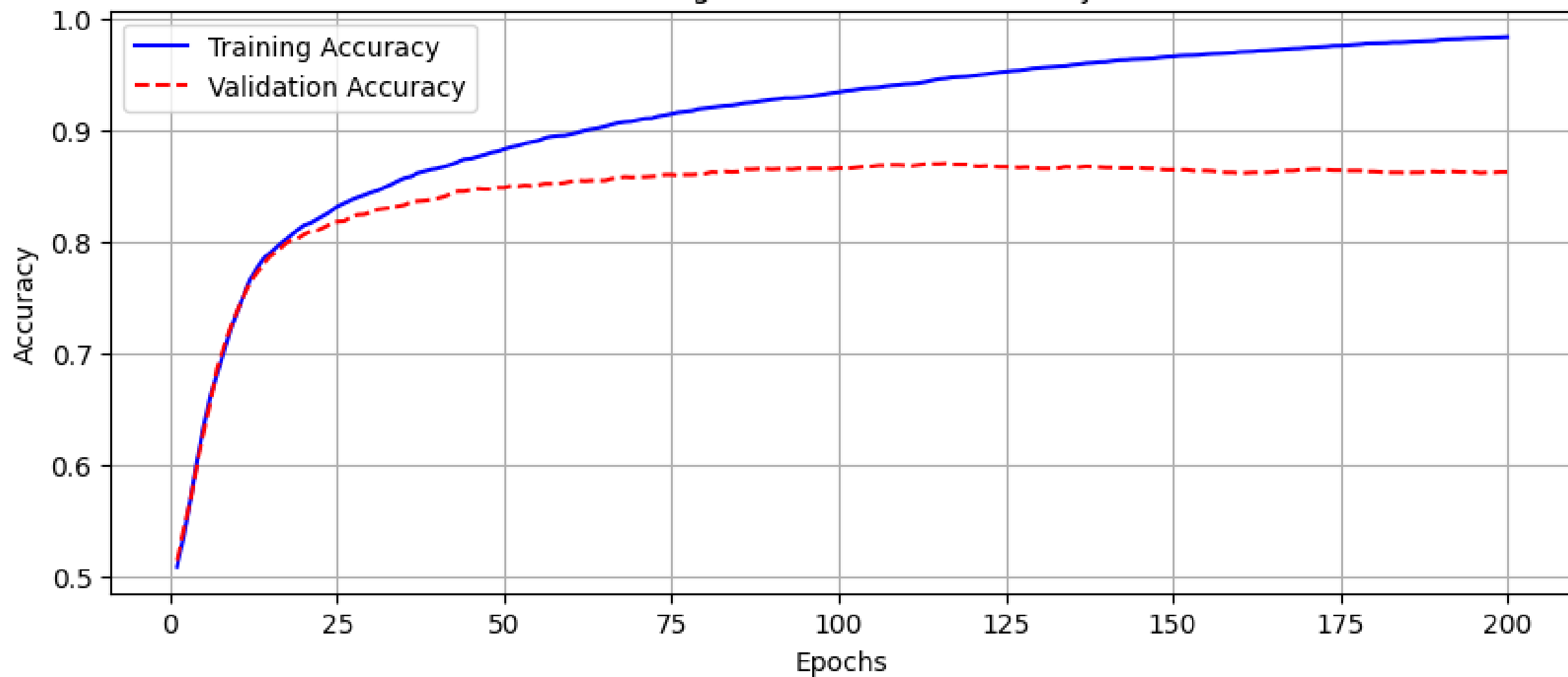
- Train error = validation error
- Perfect model?

High bias....lets increase training epochs

Low bias but high variance...



Training and Validation Accuracy



Analysis of bias and variance

- Model complexity
- Training Dataset size
- Regularization

Assume train data size is fixed...

- Increase the model complexity (#parameters)
- We will increase the number of nodes in hidden layers

```
# Suppose x_train, y_train are your training data
subset_size = int(0.5 * len(x_train_ohe)) # 50% of training data
x_train_subset = x_train_ohe[:subset_size]
y_train_subset = y_train[:subset_size]
```

```
print("Training feature shape:", x_train_subset.shape)
print("Training label shape:", y_train_subset.shape)
```

```
Training feature shape: (12500, 10000)
Training label shape: (12500,)
```

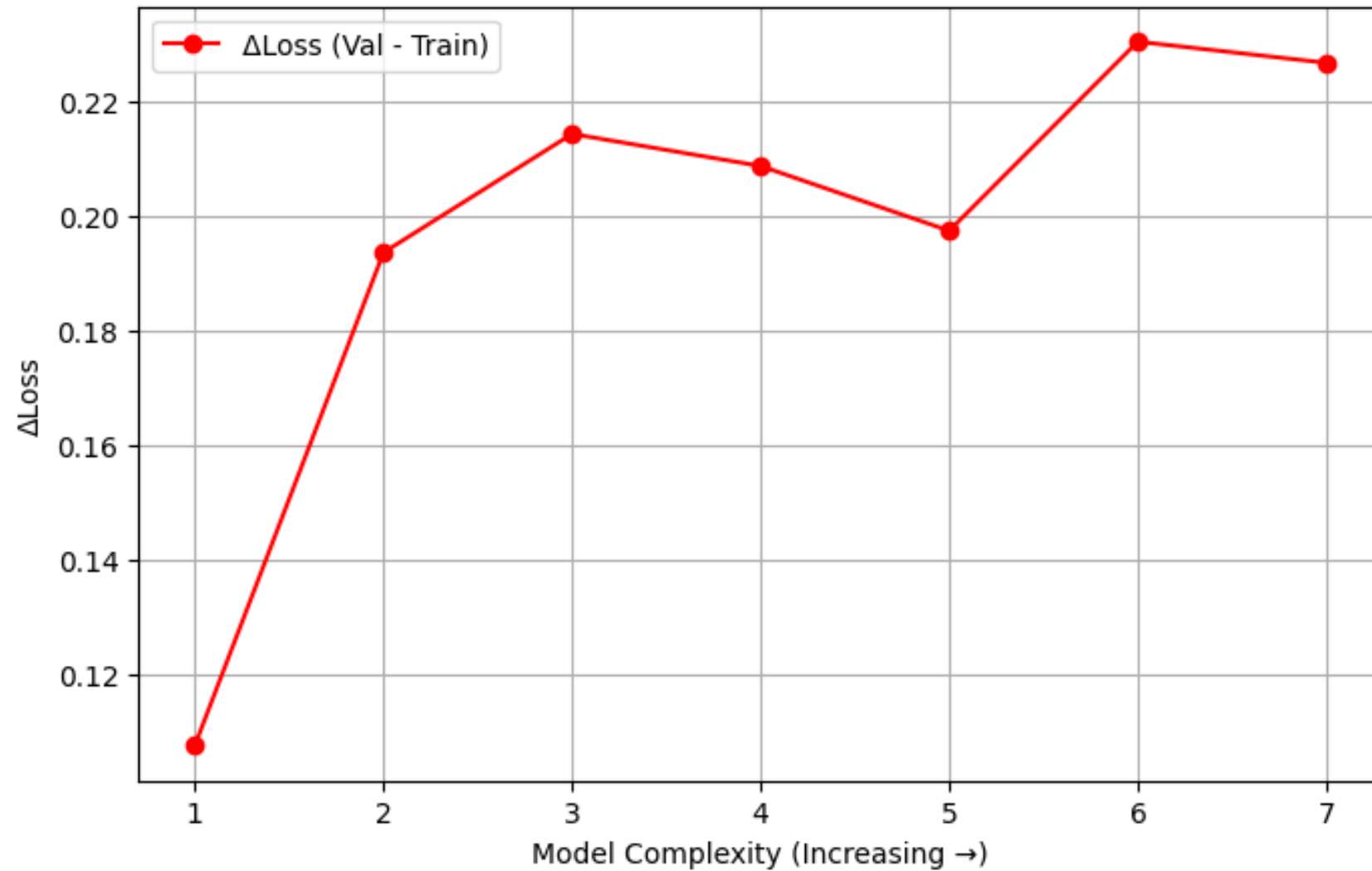
Model	Hidden Layers (units)	Formula	Total Parameters
1	(8, 4)	$(10\,000 \times 8 + 8) + (8 \times 4 + 4) + (4 \times 1 + 1)$	80 045
2	(16, 8)	$(10\,000 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 1 + 1)$	160 153
3	(32, 16)	$(10\,000 \times 32 + 32) + (32 \times 16 + 16) + (16 \times 1 + 1)$	320 561
4	(64, 32)	$(10\,000 \times 64 + 64) + (64 \times 32 + 32) + (32 \times 1 + 1)$	641 345
5	(128, 64)	$(10\,000 \times 128 + 128) + (128 \times 64 + 64) + (64 \times 1 + 1)$	1 282 945
6	(256, 128)	$(10\,000 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 1 + 1)$	2 566 273
7	(512, 256)	$(10\,000 \times 512 + 512) + (512 \times 256 + 256) + (256 \times 1 + 1)$	5 133 569

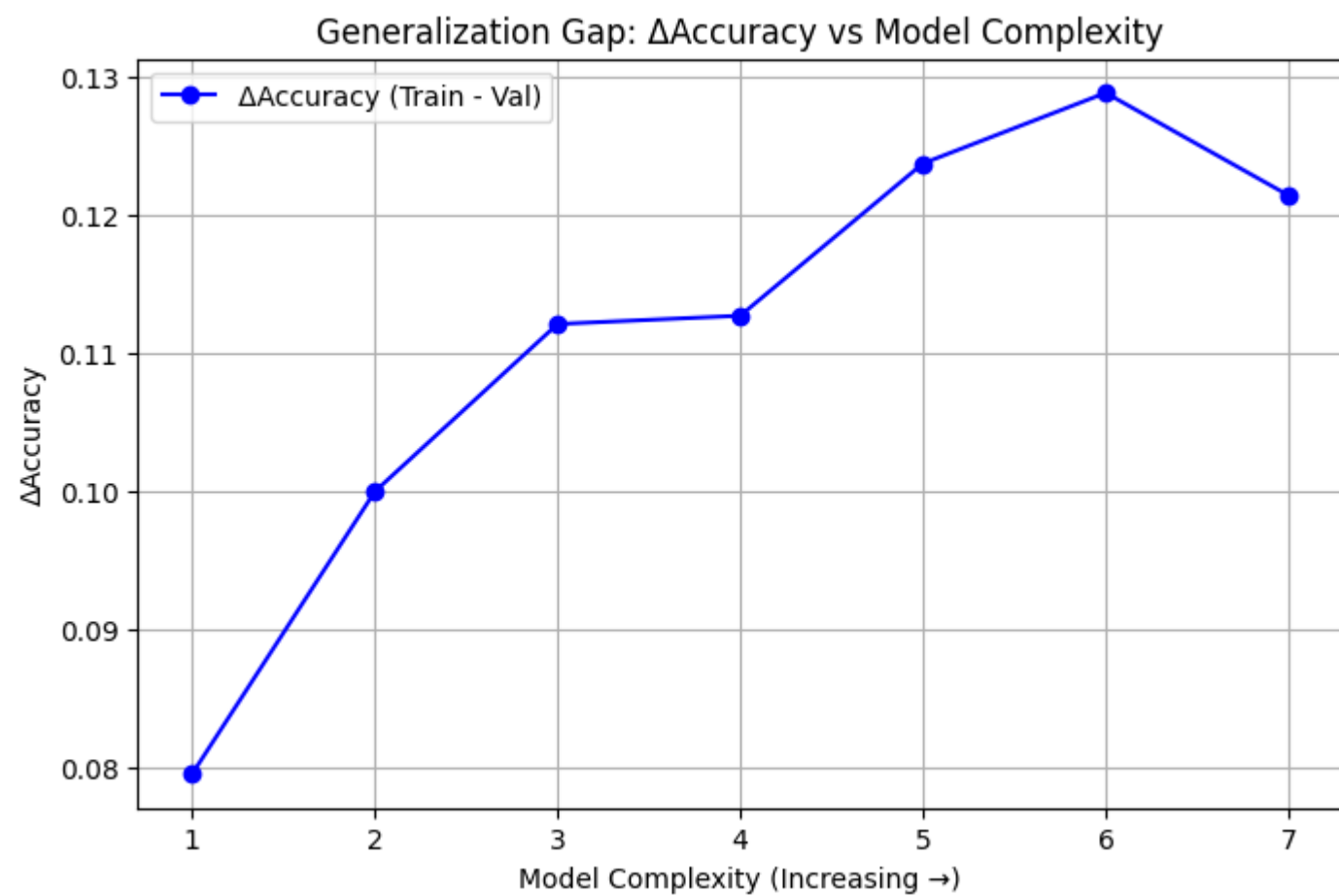


Model	Complexity	Parameters	Comment
1	Small	80 k	Tiny network, very low capacity
3	Medium	320 k	Balanced, good baseline
5	Large	1.28 M	Moderate overfitting risk
7	Very large	5.13 M	High-capacity model, prone to overfitting on small IMDB subsets

Model No.	Complexity ↑	Training Accuracy	Validation Accuracy	Train–Val Gap	Training Loss	Validation Loss (C
1	Low	0.9228	0.8432	0.0796	0.2849	0.3926
2	↑	0.9664	0.8464	0.1200	0.1558	0.3795
3	↑↑	0.9601	0.8560	0.1041	0.1837	0.3681
4	↑↑↑	0.9523	0.8496	0.1027	0.1989	0.3777
5	↑↑↑↑	0.9549	0.8512	0.1037	0.1858	0.3733
6	↑↑↑↑↑	0.9304	0.8416	0.0888	0.2417	0.3822
7	Highest	0.9574	0.8560	0.1014	0.1981	0.3749

Generalization Gap: ΔLoss vs Model Complexity





Home work

Keeping model complexity fixed, increase the data size (training).

California Housing dataset

1. Origin of the Dataset

- Source: U.S. Census data from **California in the 1990s**.
- Goal: Predict the **median house value** in a district based on various neighborhood characteristics.
- Each data sample represents a **block group** (a small geographic area with about 600–3,000 people).

The **California Housing dataset** is a *regression problem* because the model's goal is to predict a **continuous numeric output (house value)**, not to classify data into discrete categories.

Feature Name	Description	Type
MedInc	Median income in block group (in \$10,000s)	Continuous
HouseAge	Median age of houses	Continuous
AveRooms	Average number of rooms per household	Continuous
AveBedrms	Average number of bedrooms per household	Continuous
Population	Total population of the block group	Continuous
AveOccup	Average number of occupants per household	Continuous
Latitude	Latitude of the block group	Continuous
Longitude	Longitude of the block group	Continuous

```
from sklearn.datasets import fetch_california_housing

# Load the dataset
data = fetch_california_housing()

# Print feature names
print(data.feature_names)

# Print the first data point and its target
print("Features:", data.data[0])
print("Target (Median House Value):", data.target[0])
```

```
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
```

```
Features: [8.3252, 41.0, 6.9841, 1.0238, 322.0, 2.5556, 37.88, -122.23]
```

```
Target (Median House Value): 4.526
```

TensorFlow/Keras example for Predicting House Prices using the California Housing dataset (regression).

1. Load the dataset

- Use the California Housing dataset containing 8 numerical features (e.g., median income, house age, rooms, population, etc.) and target variable (median house value).

2. Split the data

- Divide the dataset into training, validation, and testing subsets to evaluate model performance properly.

3. Normalize or standardize features

- Scale all input features (e.g., using standardization) so that each has zero mean and unit variance.
- This helps the neural network learn efficiently.

4. Define the model

- Create a sequential neural network with:
 - Input layer matching the number of features (8)
 - One or two hidden Dense layers with ReLU activation
 - One output layer with a single neuron (linear activation) for regression.

5. Compile the model

- Use an optimizer like Adam.
- Choose a regression loss such as Mean Squared Error (MSE).
- Track Mean Absolute Error (MAE) as a metric for interpretability.

6. Train the model

- Fit the model on the training data.
- Use validation data or a validation split to monitor performance.
- Optionally use early stopping to prevent overfitting.

7. Evaluate the model

- Test the trained model on unseen test data.
- Report loss (MSE) and MAE.

8. Make predictions

- Use the trained model to predict house prices for new or test samples.

Compile (learning setup)

Aspect	California Housing (regression)	IMDB (binary classification)
Loss	<code>mse</code> (optimize squared error)	<code>binary_crossentropy</code>
Metrics	<code>mae</code> (and derive RMSE from loss)	<code>accuracy</code> (optionally precision/recall)
Output activation	<code>linear</code>	<code>sigmoid</code>
Target dtype	float (e.g., 2.35 $\rightarrow$ \$235k)	int {0,1}

Component	Regression (Calif.)	Classification (IMDB)
Input	8 scaled floats	10k-dim sparse BoW vector
Final layer	Dense(1)	Dense(1)
Activation	linear	sigmoid
Loss	MSE	BCE
Metric	MAE / RMSE	Accuracy
Label	real value	0/1
Preprocess	StandardScaler	Tokenizer → one-hot/TF-IDF

4) Build a small Dense network

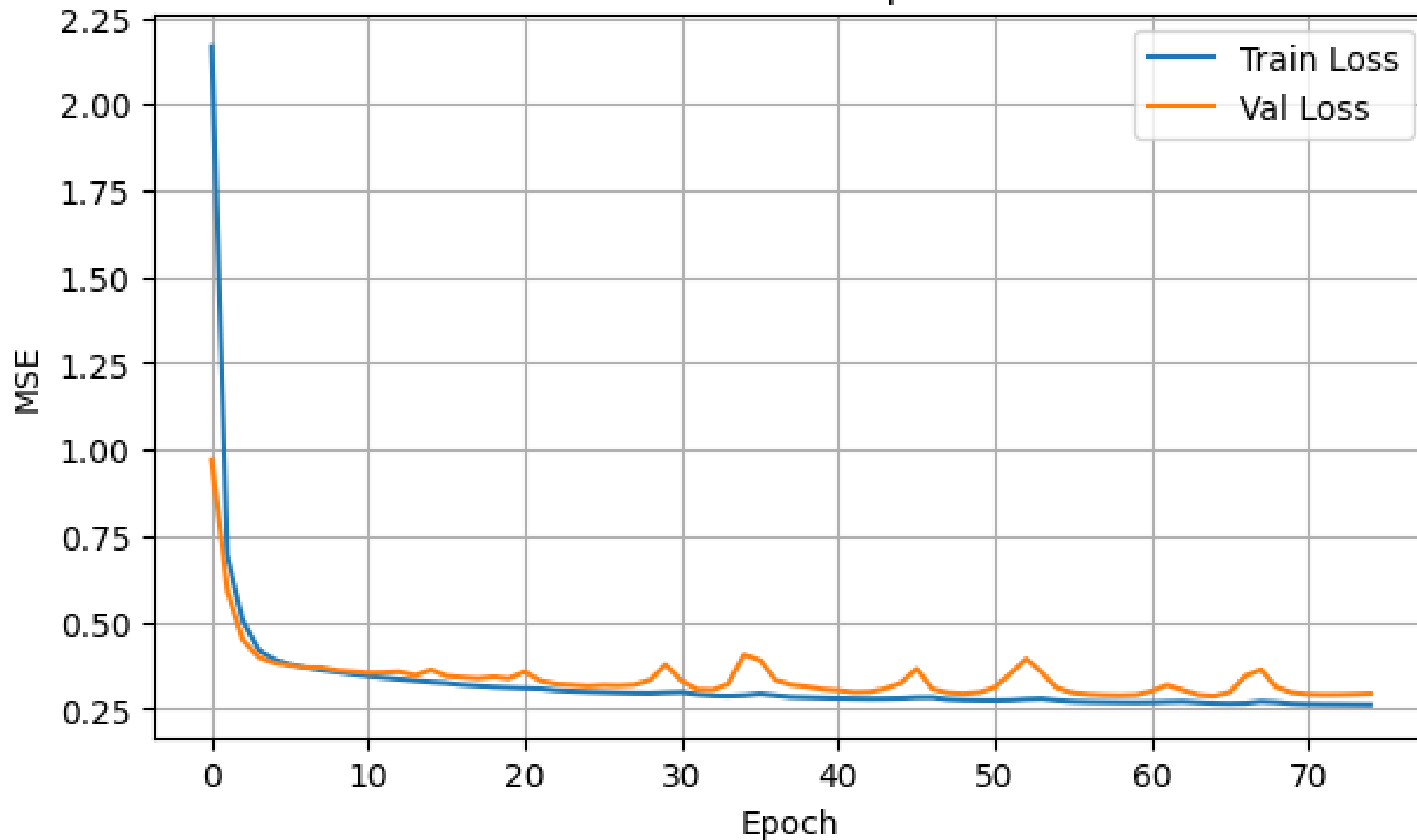
```
def build_model():  
    model = keras.Sequential([  
        layers.Input(shape=(X_train.shape[1],)),    # 8 features  
        layers.Dense(64, activation="relu"),  
        layers.Dense(32, activation="relu"),  
        layers.Dense(1, activation="linear")        # regression output  
    ])  
    model.compile(  
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),  
        loss="mse",                                # optimize MSE  
        metrics=["mae"]                             # track MAE (easy to read)  
    )  
    return model
```

```
model = build_model()
```

```
model.summary()
```

```
history = model.fit(  
    X_train, y_train,  
    validation_data=(X_val, y_val),  
    epochs=200,  
    batch_size=128,  
    verbose=1  
)
```

Loss (MSE) vs Epochs



MAE vs Epochs

